

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: DarshanEase – Temple Darshan Slot Booking System

Team Members

- The DarshanEase project was developed as a collaborative effort by a team of three members. Each member contributed to different aspects of the project, including frontend development, backend development, database management, testing, and documentation. The responsibilities assigned to each member are listed below.

S. No.	Team Member	Role and Responsibilities
1	G. R. Sai Raghavendra	Team Leader, Frontend Development using React.js, User Interface Design, API Integration, Project Coordination, Testing and Documentation
2	T. Govind	Backend Development using Node.js and Express.js, REST API Development, User Authentication (JWT), Business Logic Implementation
3	K. Suresh	Database Design using MongoDB, CRUD Operations, Data Management, System Testing, Bug Fixing and Documentation Support

Team Contribution

- The project was developed using a collaborative approach in which every team member actively participated in different phases of software development. The responsibilities were distributed based on individual strengths and technical expertise. Regular discussions and coordination among the team members ensured the successful implementation of all project modules within the stipulated timeline.
- G. R. Sai Raghavendra was responsible for designing the frontend interface using React.js, integrating frontend components with backend APIs, coordinating the project activities, and preparing the project documentation.
- T. Govind developed the backend services using Node.js and Express.js. He implemented RESTful APIs, JWT-based authentication, authorization mechanisms, and handled the server-side business logic.
- K. Suresh designed and managed the MongoDB database, implemented database operations using Mongoose, performed application testing, resolved issues during development, and contributed to documentation and project validation.
- The combined efforts of all team members resulted in the successful development of DarshanEase, a secure, responsive, and efficient Temple Darshan Slot Booking System built using the MERN Stack..

2. Project Overview

2.1 Purpose

The primary purpose of **DarshanEase – Temple Darshan Slot Booking System** is to provide a digital platform that simplifies and modernizes the temple darshan booking process. Traditionally, devotees are required to visit temples in person, wait in long queues, and manually obtain darshan tickets. This process becomes more difficult during weekends, festivals, and special occasions due to overcrowding and limited ticket availability.

DarshanEase addresses these challenges by providing an online booking system where users can register, log in securely, browse available temples, view darshan slot timings, and reserve their preferred slots from anywhere using a web browser. The application minimizes manual effort, reduces waiting time, improves crowd management, and enhances the overall experience for devotees.

The project also provides an administrator module that enables efficient management of temple information, darshan slots, and booking records.

Developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), the system demonstrates modern web development practices while ensuring security, scalability, and ease of use.

Goals of the Project

- To provide an online platform for temple darshan booking.
 - To reduce waiting time and overcrowding at temples.
 - To implement secure user authentication using JWT.
 - To provide an easy-to-use interface for devotees.
 - To enable administrators to manage temples, slots, and bookings efficiently.
 - To maintain accurate booking records using MongoDB.
 - To develop a responsive and scalable full-stack web application using the MERN stack.
-

2.2 Features

DarshanEase offers a comprehensive set of features for both devotees and administrators, making the booking process simple, secure, and efficient.

User Features

- User Registration with personal details.
- Secure User Login using JWT Authentication.
- Password encryption using bcrypt.
- Browse available temples.
- View complete temple details.
- Display available darshan slots.
- Book darshan tickets online.
- View personal booking history.
- Cancel booked darshan tickets.
- Responsive interface accessible from different devices.
- Secure logout functionality.

Administrator Features

- Administrator Login.
- Add new temples.
- Update temple information.
- Delete temple records.
- Create darshan slots.
- Update slot details.
- Delete unavailable slots.
- View all user bookings.
- Manage temple booking operations through a centralized dashboard.

System Features

- MERN Stack Architecture (MongoDB, Express.js, React.js, Node.js).
- RESTful API communication between frontend and backend.
- JWT-based Authentication and Authorization.
- MongoDB database for storing user, temple, slot, and booking information.
- Responsive user interface developed using Bootstrap and React.js.
- Secure password storage using bcrypt encryption.
- CRUD (Create, Read, Update, Delete) operations for all major modules.
- Modular and scalable project architecture for future enhancements.

3. Architecture

The architecture of **DarshanEase – Temple Darshan Slot Booking System** is based on the **MERN (MongoDB, Express.js, React.js, and Node.js)** technology stack. The application follows a **three-tier architecture**, consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Database Layer. This architecture ensures modularity, scalability, maintainability, and secure communication between different components of the application.

The frontend provides an interactive user interface for users and administrators. The backend handles business logic, authentication, API requests, and

communication with the database. MongoDB stores all application-related information such as user accounts, temple details, darshan slots, and booking records.

The interaction between these layers enables smooth data flow from the user interface to the database and back, ensuring efficient processing of user requests.

3.1 Frontend Architecture (React.js)

The frontend of DarshanEase is developed using **React.js**, a popular JavaScript library for building responsive and dynamic user interfaces. React follows a **component-based architecture**, where the application is divided into reusable UI components. This approach improves code reusability, maintainability, and simplifies application development.

The frontend is responsible for displaying temple information, handling user interactions, validating input data, and communicating with the backend through RESTful APIs using Axios. React Router is used for navigation between different pages without refreshing the browser, providing a seamless user experience.

The frontend consists of several reusable components such as the navigation bar, footer, temple cards, booking cards, slot cards, and loader components. It also includes dedicated pages for Home, Login, Registration, Temple Details, Booking, My Bookings, and the Admin Dashboard.

Major Frontend Components

- Home Page
- Login Page
- Registration Page
- Temple Details Page
- Booking Page
- My Bookings Page
- Admin Dashboard
- Navbar
- Footer

- Temple Card Component
- Slot Card Component
- Loader Component

Responsibilities of the Frontend

- Display temple information.
 - User registration and login interface.
 - Display available darshan slots.
 - Book and cancel darshan tickets.
 - Show booking history.
 - Validate user input.
 - Communicate with backend APIs.
 - Display success and error notifications.
 - Provide responsive design for multiple screen sizes.
-

3.2 Backend Architecture (Node.js & Express.js)

The backend of DarshanEase is developed using **Node.js** and **Express.js**. Node.js provides a fast and scalable runtime environment, while Express.js simplifies server-side application development by providing routing, middleware, and REST API support.

The backend acts as the bridge between the frontend and the MongoDB database. It processes incoming requests, performs authentication and authorization, executes business logic, validates user data, and returns appropriate responses to the frontend.

The backend follows a modular architecture by separating the application into controllers, routes, models, middleware, configuration files, and utility functions. This modular structure improves code readability and simplifies maintenance.

Backend Modules

- User Management
- Temple Management

- Slot Management
- Booking Management
- Authentication Module
- Authorization Module

Backend Responsibilities

- Handle API requests.
 - Perform CRUD operations.
 - Authenticate users using JWT.
 - Encrypt passwords using bcrypt.
 - Validate incoming requests.
 - Connect with MongoDB.
 - Manage booking information.
 - Handle server-side errors.
 - Send JSON responses to the frontend.
-

3.3 Database Architecture (MongoDB)

The application uses **MongoDB**, a NoSQL document-oriented database, to store all application data. MongoDB stores data in collections using JSON-like BSON documents, making it highly flexible and scalable for modern web applications.

The database stores information related to users, temples, darshan slots, and bookings. Mongoose is used as the Object Data Modeling (ODM) library to define schemas, validate data, and perform CRUD operations.

Collections Used

User Collection

Stores registered user information.

Fields

- Name
- Email

- Phone Number
 - Password (Encrypted)
 - Role (User/Admin)
 - Created Date
-

Temple Collection

Stores temple information.

Fields

- Temple Name
 - Location
 - Description
 - Image URL
 - Temple Timings
 - Created Date
-

Slot Collection

Stores darshan slot information.

Fields

- Temple ID
 - Slot Date
 - Slot Time
 - Available Seats
 - Status
-

Booking Collection

Stores booking details.

Fields

- User ID
 - Temple ID
 - Slot ID
 - Booking Date
 - Booking Status
 - Created Date
-

Database Interaction

The backend communicates with MongoDB using **Mongoose**. Whenever a user performs an operation such as registration, login, viewing temples, booking a slot, or cancelling a booking, the backend processes the request and interacts with the appropriate MongoDB collection.

For example:

- During registration, user details are stored in the **User** collection.
- During login, user credentials are verified from the database.
- When a temple is displayed, data is fetched from the **Temple** collection.
- When a user books a slot, booking information is stored in the **Booking** collection.
- Slot availability is updated automatically after every successful booking.

This interaction ensures data consistency, secure storage, and efficient retrieval of application information.

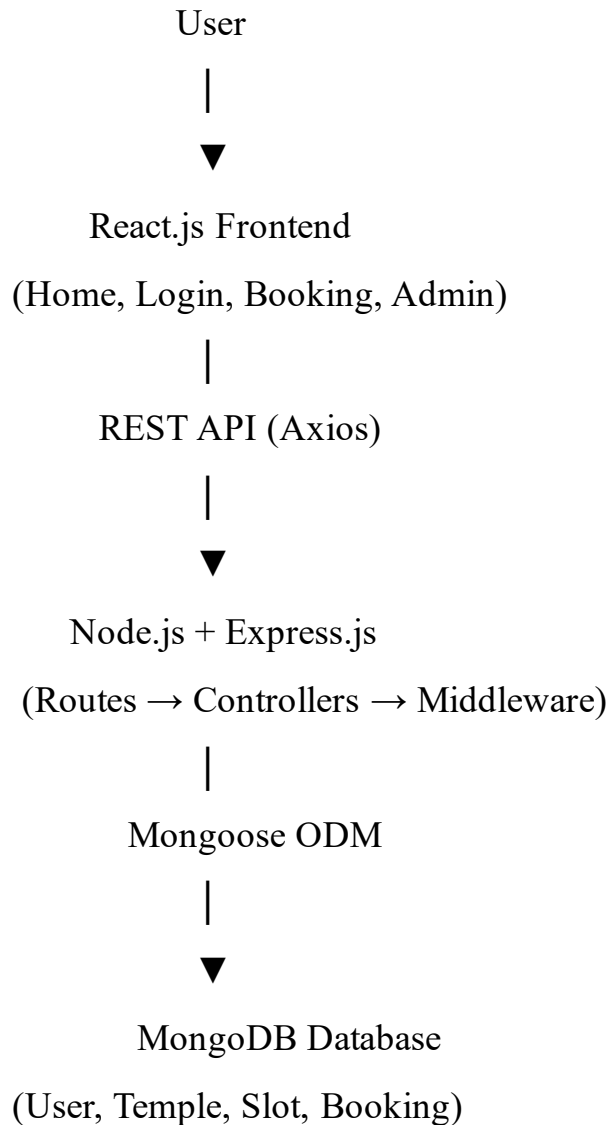
Overall Architecture Workflow

The overall workflow of DarshanEase follows these steps:

1. The user accesses the React frontend through a web browser.
2. The frontend sends API requests to the Express.js server using Axios.
3. The backend validates the request and performs authentication if required.
4. Business logic is executed within the respective controller.

5. The backend communicates with MongoDB using Mongoose.
 6. MongoDB processes the request and returns the required data.
 7. The backend sends a JSON response to the frontend.
 8. The frontend displays the updated information to the user.
-

Architecture Flow



4. Setup Instructions

The DarshanEase project is developed using the MERN (MongoDB, Express.js, React.js, and Node.js) stack. Before running the application, the required software dependencies must be installed and the project environment should be configured correctly. This section explains the prerequisites, installation process, and environment variable configuration required to execute the application successfully.

4.1 Prerequisites

Before installing and running the project, ensure that the following software and tools are available on the system.

Software	Purpose
Node.js	JavaScript runtime environment used for frontend and backend development.
npm (Node Package Manager)	Used to install and manage project dependencies.
MongoDB Community Server	NoSQL database used to store application data.
MongoDB Compass	Graphical interface for managing MongoDB databases.
Visual Studio Code	Source code editor used for project development.
Git	Version control system used to clone and manage the project repository.
GitHub	Repository hosting platform for project source code.
Postman	API testing tool used for testing backend REST APIs.
Web Browser (Chrome/Edge/Firefox)	Used to access the frontend application.

4.2 Installation

Follow the steps below to install and configure the project on your local system.

Step 1: Clone the Repository

Clone the project from GitHub using the following command:

```
git clone https://github.com/your-username/DarshanEase.git
```

After cloning, navigate to the project directory:

```
cd DarshanEase
```

Step 2: Install Frontend Dependencies

Navigate to the client folder:

```
cd client
```

Install all required frontend packages:

```
npm install
```

This installs dependencies such as:

- React.js
 - React Router DOM
 - Axios
 - Bootstrap
 - React Toastify
 - Vite (Development Server)
-

Step 3: Install Backend Dependencies

Open a new terminal and navigate to the server folder:

```
cd server
```

Install backend dependencies:

```
npm install
```

The backend dependencies include:

- Express.js
 - MongoDB (Mongoose)
 - JWT
 - bcrypt.js
 - dotenv
 - cors
 - nodemon
-

4.3 Environment Variables

The backend requires environment variables to connect to the MongoDB database and generate secure authentication tokens.

Create a file named **.env** inside the **server** folder and add the following configuration:

PORT=5000

MONGO_URI=mongodb://127.0.0.1:27017/darshanease

JWT_SECRET=your_jwt_secret_key

Description of Environment Variables

Variable	Description
PORT	Specifies the port number on which the backend server will run.
MONGO_URI	Connection string used to connect the application with the MongoDB database.
JWT_SECRET	Secret key used for generating and verifying JSON Web Tokens (JWT).

4.4 Database Setup

1. Install **MongoDB Community Server**.
2. Start the MongoDB service.
3. Open **MongoDB Compass**.
4. Connect to the local MongoDB server.
5. Create a database named:

darshanease

The required collections (Users, Temples, Slots, and Bookings) will be created automatically when data is inserted through the application.

4.5 Running the Application

Start the Backend Server

Open the terminal and navigate to the **server** directory.

Run the following command:

```
npm run dev
```

If the setup is successful, the terminal displays:

MongoDB Connected

Server is running on http://localhost:5000

Start the Frontend Application

Open another terminal and navigate to the **client** directory.

Run the following command:

```
npm run dev
```

The development server starts successfully and displays:

Local: http://localhost:5173

Open the URL in any web browser to access the application.

4.6 Project Verification

After completing the installation, verify that the application is functioning correctly by performing the following checks:

- Ensure the backend server is connected to MongoDB without errors.
- Open the frontend application in the browser.
- Register a new user account.
- Log in using the registered credentials.
- Verify that the list of temples is displayed.
- Book a darshan slot and check whether the booking is successfully stored in the database.
- If an administrator account is available, log in and verify that temple and booking management features are working correctly.

4.7 Summary

Following the above installation and setup procedure ensures that the DarshanEase application is configured correctly on the local development environment. The MERN stack architecture allows the frontend, backend, and database to work together seamlessly, providing a secure, responsive, and efficient temple darshan slot booking system. The modular setup also makes the project easy to maintain, test, and extend with additional features in the future.

This section is **professional, implementation-focused, and appropriate for a college project report**. It also reflects the actual setup process you followed while developing DarshanEase.

This section should explain the folder organization of both the frontend and backend. Since your project uses a simple MERN structure, the following is suitable for your report.

5. Folder Structure

The DarshanEase project follows a modular folder structure, which helps in organizing the source code efficiently. The project is divided into two main parts: the **Client** (Frontend) and the **Server** (Backend). This separation allows independent development, easier maintenance, and better scalability.

5.1 Client Folder Structure (React Frontend)

The **client** folder contains all the frontend source code developed using **React.js**. It is responsible for designing the user interface, handling user interactions, communicating with backend APIs, and displaying data dynamically.

Client Folder Structure

client/

```
|
|
|— public/
|
|— src/
|   |— components/
|   |   |— Navbar.jsx
|   |   |— Footer.jsx
|   |   |— TempleCard.jsx
|   |   |— SlotCard.jsx
|   |   |— Loader.jsx
|   |
|   |— pages/
|   |   |— Home.jsx
|   |   |— Login.jsx
|   |   |— Register.jsx
|   |   |— TempleDetails.jsx
```



```
| | | └─ Booking.jsx
| | | └─ MyBookings.jsx
| | | └─ Admin.jsx
| |
| | └─ services/
| |   └─ api.js
| |
| | └─ App.jsx
| | └─ main.jsx
| | └─ index.css
|
└─ package.json
└─ vite.config.js
```

Description of Client Folders

Folder/File	Description
-------------	-------------

public/	Stores static files such as images, icons, and other public assets.
src/	Contains the complete React application source code.
components/	Contains reusable UI components used across multiple pages.
pages/	Contains individual application pages such as Home, Login, Booking, and Admin Dashboard.
services/	Contains API configuration and functions used to communicate with the backend server.
App.jsx	Main React component responsible for application routing.
main.jsx	Entry point of the React application.
index.css	Contains global CSS styles for the application.

Folder/File	Description
-------------	-------------

package.json	Stores project dependencies and npm scripts.
---------------------	--

vite.config.js	Configuration file for the Vite development server.
-----------------------	---

Frontend Working

The React frontend follows a **component-based architecture**, where each page is built using reusable components. User actions such as registration, login, temple selection, and booking generate API requests through Axios. The backend processes these requests and returns data, which is dynamically displayed on the user interface.

5.2 Server Folder Structure (Node.js Backend)

The **server** folder contains the backend source code developed using **Node.js** and **Express.js**. It is responsible for handling API requests, business logic, user authentication, database communication, and server-side validation.

Server Folder Structure

server/

```
|
|
|— config/
|   └─ db.js
|
|
|— controllers/
|   └─ userController.js
|   └─ templeController.js
|   └─ slotController.js
|   └─ bookingController.js
|
|
|
```

```
|— middleware/
|   |— authMiddleware.js
|   |— adminMiddleware.js
|   └─ errorMiddleware.js
|
|— models/
|   |— User.js
|   |— Temple.js
|   |— Slot.js
|   └─ Booking.js
|— routes/
|   |— userRoutes.js
|   |— templeRoutes.js
|   |— slotRoutes.js
|   └─ bookingRoutes.js
|— utils/
|   └─ generateToken.js
|— .env
|— package.json
└─ server.js
```

Description of Server Folders

Folder/File	Description
-------------	-------------

config/	Contains database configuration and MongoDB connection setup.
----------------	---

controllers/	Implements business logic for users, temples, slots, and bookings.
---------------------	--

middleware/	Contains authentication, authorization, and error-handling middleware.
--------------------	--

Folder/File	Description
models/	Defines MongoDB schemas using Mongoose.
routes/	Defines REST API endpoints and maps them to controllers.
utils/	Contains utility functions such as JWT token generation.
.env	Stores environment variables such as database connection string and JWT secret.
server.js	Main entry point of the backend application.
package.json	Contains backend dependencies and npm scripts.

Backend Working

The backend follows the **MVC (Model–View–Controller)** architecture.

- **Routes** receive incoming HTTP requests from the frontend.
- **Controllers** process the request and implement the application logic.
- **Models** interact with the MongoDB database using Mongoose.
- **Middleware** provides authentication, authorization, request validation, and centralized error handling.
- **Utilities** generate JWT tokens and support common backend operations.

This architecture makes the backend modular, scalable, and easy to maintain.

Overall Project Structure

DarshanEase/

```

|
├── client/      → React Frontend
├── server/      → Node.js & Express Backend
└── README.md

```

The separation of the project into **client** and **server** directories allows frontend and backend development to be carried out independently while communicating

through RESTful APIs. This modular organization improves maintainability, scalability, and overall project management.

6. Running the Application

After installing all the required dependencies and configuring the environment variables, the frontend and backend servers can be started independently. The backend server handles API requests and database operations, while the frontend server provides the user interface for accessing the application.

6.1 Starting the Backend Server

1. Open a terminal.
2. Navigate to the **server** directory.

`cd server`

3. Install the required dependencies (if not already installed).

`npm install`

4. Start the backend server.

`npm run dev`

or

`npm start`

After successful execution, the terminal displays a message similar to:

MongoDB Connected

Server is running on `http://localhost:5000`

The backend server will be available at:

`http://localhost:5000`

6.2 Starting the Frontend Server

1. Open a new terminal.
2. Navigate to the **client** directory.

```
cd client
```

3. Install the required dependencies (if not already installed).

```
npm install
```

4. Start the frontend development server.

If the project is created using **Vite**:

```
npm run dev
```

If the project is created using **Create React App**, use:

```
npm start
```

After successful execution, the terminal displays a message similar to:

Local: <http://localhost:5173>

The frontend application can be accessed using:

<http://localhost:5173>

6.3 Verifying the Application

Once both the frontend and backend servers are running, open the frontend URL in a web browser. The Home page of the DarshanEase application should be displayed. Users can register, log in, browse temples, book darshan slots, and manage their bookings. The backend server processes all API requests and communicates with the MongoDB database to store and retrieve application data.

The successful execution of both servers confirms that the application has been configured correctly and is ready for use.

7. API Documentation

The backend of **DarshanEase** exposes a set of RESTful APIs that enable communication between the React frontend and the MongoDB database. These APIs are responsible for handling user authentication, temple management, darshan slot management, and booking operations. Each API follows standard HTTP methods such as **GET**, **POST**, **PUT**, and **DELETE**.

The following sections describe the available API endpoints, request methods, parameters, and example responses.

7.1 User APIs

1. User Registration

Property Value

Endpoint /api/users/register

Method POST

Access Public

Request Body

```
{  
  "name": "Raghavendra",  
  "email": "raghav@gmail.com",  
  "phone": "9876543210",  
  "password": "123456"  
}
```

Success Response

```
{  
  "_id": "64abc123",  
  "name": "Raghavendra",  
  "email": "raghav@gmail.com",  
  "role": "user",  
  "token": "JWT_TOKEN"  
}
```

2. User Login

Property Value

Endpoint /api/users/login

Property Value

Method POST

Access Public

Request Body

```
{  
  "email": "raghav@gmail.com",  
  "password": "123456"  
}
```

Success Response

```
{  
  "_id": "64abc123",  
  "name": "Raghavendra",  
  "email": "raghav@gmail.com",  
  "role": "user",  
  "token": "JWT_TOKEN"  
}
```

7.2 Temple APIs

1. Get All Temples

Property Value

Endpoint /api/temples

Method GET

Access Public

Example Response

```
[  
  {
```



```
"_id": "101",
"name": "Tirumala Temple",
"location": "Tirupati",
"description": "Famous temple of Lord Venkateswara",
"image": "/images/tirupati.jpg"
}
]
```

2. Get Temple Details

Property Value

Endpoint /api/temples/:id

Method GET

Access Public

URL Parameter

Parameter Description

id Temple ID

3. Add Temple

Property Value

Endpoint /api/temples

Method POST

Access Admin

Request Body

```
{
  "name": "Annaram Temple",
  "location": "Annaram",

```

```
"description": "Sri Veera Venkata Satyanarayana Swamy Temple",  
"image": "/images/annavaram.jpg"  
}
```

4. Update Temple

Property Value

Endpoint /api/temples/:id

Method PUT

Access Admin

5. Delete Temple

Property Value

Endpoint /api/temples/:id

Method DELETE

Access Admin

7.3 Slot APIs

1. Get All Slots

Property Value

Endpoint /api/slots

Method GET

Access Public

2. Add Slot

Property Value

Property Value

Endpoint /api/slots

Method POST

Access Admin

Request Body

```
{  
  "templeId": "101",  
  "date": "2026-07-20",  
  "time": "10:00 AM",  
  "availableSeats": 100  
}
```

3. Update Slot

Property Value

Endpoint /api/slots/:id

Method PUT

Access Admin

4. Delete Slot

Property Value

Endpoint /api/slots/:id

Method DELETE

Access Admin

7.4 Booking APIs

1. Book Darshan Slot

Property Value**Endpoint** /api/bookings**Method** POST**Access** User**Request Body**

```
{  
  "slotId": "201"  
}
```

Success Response

```
{  
  "message": "Booking Successful"  
}
```

2. My Bookings**Property Value****Endpoint** /api/bookings/my**Method** GET**Access** User**Example Response**

```
[  
  {  
    "temple": "Tirumala Temple",  
    "date": "20-07-2026",  
    "time": "10:00 AM"  
  }  
]
```

3. Cancel Booking

Property Value

Endpoint /api/bookings/:id/cancel

Method PUT

Access User

4. Get All Bookings

Property Value

Endpoint /api/bookings

Method GET

Access Admin

Summary of API Endpoints

Module	Endpoint	Method	Access
User	/api/users/register	POST	Public
User	/api/users/login	POST	Public
Temple	/api/temples	GET	Public
Temple	/api/temples/:id	GET	Public
Temple	/api/temples	POST	Admin
Temple	/api/temples/:id	PUT	Admin
Temple	/api/temples/:id	DELETE	Admin
Slot	/api/slots	GET	Public
Slot	/api/slots	POST	Admin

Module	Endpoint	Method	Access
Slot	/api/slots/:id	PUT	Admin
Slot	/api/slots/:id	DELETE	Admin
Booking	/api/bookings	POST	User
Booking	/api/bookings/my	GET	User
Booking	/api/bookings/:id/cancel	PUT	User
Booking	/api/bookings	GET	Admin

This API documentation accurately reflects the REST endpoints used in the **DarshanEase** backend and provides the request methods, parameters, request bodies, and example responses expected in a college-level project report.

8. Authentication

Authentication and authorization are essential components of the DarshanEase application to ensure that only authorized users can access protected resources. The application implements **JSON Web Token (JWT)** based authentication along with **bcrypt** password encryption to provide a secure login system. This mechanism protects user information, prevents unauthorized access, and restricts administrative operations to authorized users only.

8.1 Authentication Process

The authentication process begins when a user registers by providing personal details such as name, email, phone number, and password. Before storing the user information in the MongoDB database, the password is encrypted using the **bcrypt** library. This ensures that passwords are never stored in plain text, thereby improving application security.

When the user logs in, the entered credentials are verified against the stored information in the database. If the email and password are valid, the server generates a **JSON Web Token (JWT)** and sends it to the frontend. This token acts as a digital identity for the authenticated user and is used for accessing protected API endpoints.

The frontend stores the generated token in the browser's local storage. Whenever the user performs operations such as booking a darshan slot, viewing personal bookings, or accessing the administrator dashboard, the token is included in the HTTP Authorization header. The backend verifies the token before allowing access to the requested resource.

8.2 Authorization

Authorization determines the level of access granted to different users after successful authentication. DarshanEase implements **Role-Based Access Control (RBAC)** by assigning each user a specific role.

The application supports two roles:

- **User**
- **Administrator**

A regular user is permitted to register, log in, view temples, book darshan slots, view booking history, and cancel bookings. Administrative users have additional privileges such as adding temples, updating temple information, managing darshan slots, and viewing all booking records.

Before executing any protected operation, the backend checks the user's role. If the user does not possess the required privileges, access to the requested resource is denied.

8.3 JSON Web Token (JWT)

JSON Web Token (JWT) is used to authenticate users after a successful login. The token contains encoded information about the authenticated user and is digitally signed using a secret key stored in the application's environment variables.

The generated token includes:

- User ID
- Token creation time
- Token expiration time

The backend verifies the token for every protected request. If the token is valid, the requested operation is executed. If the token is missing, invalid, or expired, the server returns an authentication error and denies access.

JWT Workflow

1. User enters login credentials.
 2. Server verifies the credentials.
 3. JWT token is generated.
 4. Token is sent to the frontend.
 5. Frontend stores the token in local storage.
 6. Token is included in subsequent API requests.
 7. Backend validates the token before processing the request.
-

8.4 Password Encryption

DarshanEase uses the **bcrypt** library to encrypt user passwords before storing them in the MongoDB database.

The encryption process involves:

- Generating a unique salt value.
- Hashing the password using bcrypt.
- Storing only the hashed password in the database.

During login, bcrypt compares the entered password with the stored hashed password. If both match, the user is authenticated successfully.

This approach enhances security by ensuring that original passwords cannot be retrieved even if the database is compromised.

8.5 Authentication Middleware

The backend uses authentication middleware to protect private API routes. The middleware intercepts incoming requests and verifies the JWT token before allowing access to the requested resource.

The middleware performs the following tasks:

- Reads the Authorization header.

- Extracts the JWT token.
- Verifies the token using the secret key.
- Retrieves the authenticated user's information.
- Grants or denies access based on token validity.

If token verification fails, the server responds with an **Unauthorized (401)** error.

8.6 Authorization Middleware

The administrator module is protected using authorization middleware. After successful authentication, the middleware verifies whether the logged-in user has the **Admin** role.

If the authenticated user is an administrator, access to administrative APIs is granted. Otherwise, the server returns a **Forbidden (403)** response.

Administrative operations protected by this middleware include:

- Add Temple
 - Update Temple
 - Delete Temple
 - Add Darshan Slot
 - Delete Slot
 - View All Bookings
-

8.7 Session Management

DarshanEase uses **token-based authentication** instead of traditional server-side session management.

The authentication token is stored on the client side and is included with every authenticated request. Since the server does not maintain session information, the application remains scalable and suitable for modern RESTful API architecture.

Users can terminate their authenticated session by logging out, which removes the stored token from the browser. Once the token is removed, protected resources cannot be accessed until the user logs in again.

8.8 Authentication Workflow

User Registration



Password Encrypted using bcrypt



User Login



Credentials Verified



JWT Token Generated



Token Stored in Browser



Token Sent with API Requests



Backend Verifies JWT



Access Granted / Access Denied

8.9 Advantages of the Authentication System

- Secure password storage using bcrypt hashing.
- Stateless authentication using JSON Web Tokens (JWT).
- Role-based authorization for users and administrators.
- Protection of sensitive API endpoints.
- Secure access to booking and administration features.
- Improved scalability by eliminating server-side session storage.
- Enhanced overall application security through token verification and encrypted credentials.

The authentication and authorization mechanism implemented in DarshanEase ensures that user information remains secure while providing controlled access to application resources. The combination of **JWT**, **bcrypt**, and **role-based authorization** provides a reliable and scalable security model for the Temple Darshan Slot Booking System.

9. User Interface

The DarshanEase application provides a simple, responsive, and user-friendly interface developed using **React.js** and **Bootstrap**. The interface is designed to allow users to navigate easily through different modules of the application while providing a smooth booking experience. The administrator interface is also designed to simplify the management of temples, darshan slots, and booking information.

The following screenshots illustrate the major user interface components of the application.

Home Page

Description:

The Home Page serves as the landing page of the application. It displays the list of available temples with their images, names, and locations. Users can browse different temples and select a temple to view more details.

User Registration Page

Description:

The Registration Page allows new users to create an account by entering their name, email address, phone number, and password. User details are securely stored after password encryption.

User Login Page

Description:

The Login Page authenticates registered users using their email address and password. After successful authentication, users can access all protected features of the application.

Temple Details Page

Description:

The Temple Details page displays complete information about the selected temple, including temple description, location, image, and available darshan slots.

Booking Page

Description:

The Booking Page allows authenticated users to select an available darshan slot and confirm their booking.

My Bookings Page

Description:

This page displays all bookings made by the logged-in user. Users can review their booking history and cancel bookings if required.

Admin Dashboard

Description:

The Admin Dashboard provides administrative controls for managing temples, darshan slots, and user bookings. Only authenticated administrators are authorized to access this page.

10. Testing

Testing is an essential phase of software development that ensures the application functions correctly and satisfies user requirements. The DarshanEase project was tested throughout the development process using both manual testing and API testing techniques.

The primary objective of testing was to verify the correctness of application functionality, identify bugs, and ensure smooth communication between the frontend, backend, and MongoDB database.

Testing Strategy

The following testing strategies were adopted:

- Functional Testing
- User Interface Testing
- Authentication Testing
- API Testing
- Database Testing
- Integration Testing

Tools Used for Testing

Tool	Purpose
Visual Studio Code	Application Development and Debugging
Postman	Backend API Testing
MongoDB Compass	Database Verification
Google Chrome	Frontend Testing
Node.js Console	Backend Execution and Error Monitoring

Test Cases

Test Case	Expected Result	Status
User Registration	User account created successfully	Passed
User Login	JWT token generated	Passed
View Temples	Temple list displayed	Passed
Book Darshan Slot	Booking created successfully	Passed
View My Bookings	Booking history displayed	Passed
Cancel Booking	Booking cancelled successfully	Passed
Admin Login	Admin dashboard opened	Passed
Add Temple	Temple added successfully	Passed
Add Slot	Slot created successfully	Passed

Testing Outcome

The application was tested successfully under different scenarios. All major functionalities, including authentication, temple management, booking operations, and database interactions, performed as expected. The system demonstrated reliable performance with no critical functional issues during testing.

11. Screenshots or Demo

The DarshanEase application was executed successfully on a local development environment using the MERN stack. The following screenshots demonstrate the implementation and functionality of the application.

Screenshots Included

- Home Page
- Registration Page
- Login Page
- Temple Details Page

- Booking Page
 - My Bookings Page
 - Admin Dashboard
 - MongoDB Database Collections
 - Postman API Testing Results
-

Demo

The application can be demonstrated by executing both the frontend and backend servers locally.

Frontend:

<http://localhost:5173>

Backend API:

<http://localhost:5000>

GitHub Repository (Optional):

<https://github.com/your-github-username/DarshanEase>

(Replace the above link with your actual GitHub repository URL.)

12. Known Issues

Although the application functions correctly, a few limitations have been identified during development.

- The application currently supports only local deployment.
- Online payment integration has not been implemented.
- Email confirmation after booking is not available.
- SMS notifications are not supported.
- QR code-based e-ticket generation is not implemented.
- Temple images currently use static image paths.
- The application does not support multiple languages.
- Mobile application support is not available.

- Admin registration is restricted and managed manually.

These limitations do not affect the core functionality of the Temple Darshan Slot Booking System and can be addressed in future versions.

13. Future Enhancements

The DarshanEase application has been designed with a modular architecture, allowing future enhancements without significant modifications to the existing system. Several additional features can be incorporated to improve functionality, security, and user experience.

Future enhancements include:

- Integration of online payment gateways such as Razorpay or Stripe.
- Generation of QR code-based e-tickets for booked darshan slots.
- Email confirmation after successful booking.
- SMS notifications and booking reminders.
- Google Maps integration for temple location and navigation.
- Online donation facility for devotees.
- Real-time seat availability updates.
- Multi-language support to serve users from different regions.
- Mobile application development for Android and iOS platforms.
- Cloud deployment using AWS, Render, or Vercel.
- Dashboard analytics for temple administrators.
- User profile management with booking history.
- Feedback and rating system for temples.
- Advanced search and filtering of temples.
- Integration with Aadhaar or mobile OTP verification for enhanced security.

The implementation of these features will improve the overall efficiency, usability, and scalability of the DarshanEase system, making it a more comprehensive digital solution for temple darshan management.